

Tut07

Anne

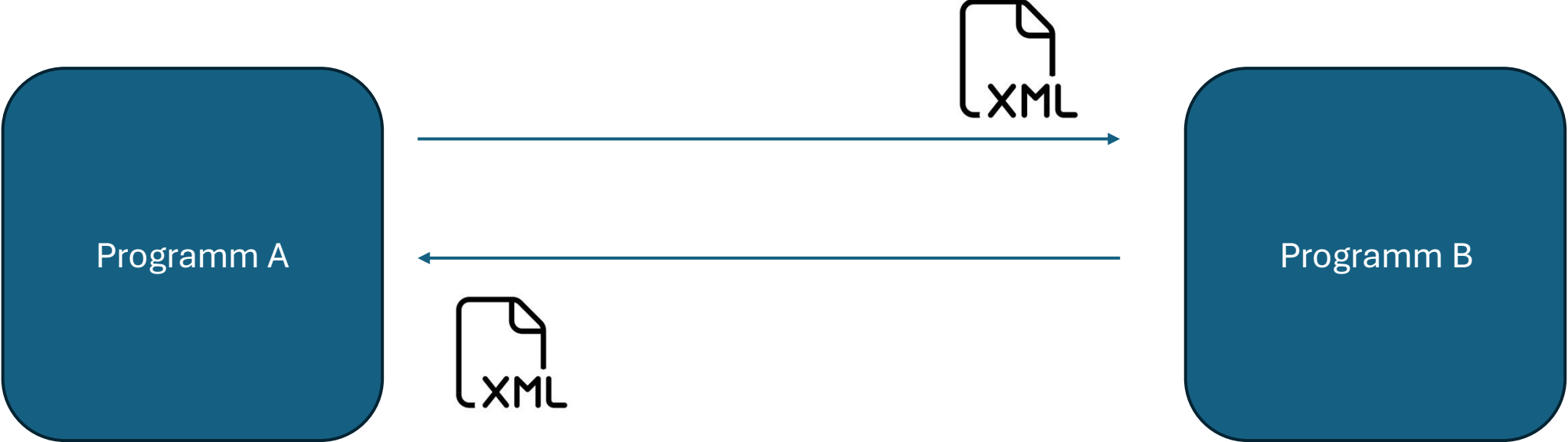
TODO's

XML-Schema

Protobuf

Datenbanken

SQL – ganz Basic



XSD (XML-Schema)

- Regelwerk, dass Struktur, Inhalt und Datentypen von XML-Datei beschreibt
- Wie Schablone
- Soll sicherstellen, dass Programm fehlerfrei verarbeitet wird

1st immer root
element

```
<?xml ...>
<xs:schema ...>
  <xs:simpleType name="termType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="summer term"/>
      <xs:enumeration value="winter term"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="examType">
    <xs:sequence>
      <xs:element name="lecture" type="xs:string" maxOccurs="1"/>
      <xs:element name="term" type="termType" maxOccurs="1"/>
    </xs:sequence>
    <xs:attribute name="room" type="xs:string"/>
  </xs:complexType>
  <xs:complexType name="examListType">
    <xs:sequence>
      <xs:element name="exam" type="examType" maxOccurs="Unbounded"/>
    </xs:sequence>
  </xs:complexType>
  <xs:element name="exams" type="examListType"/>
</xs:schema>
```

XSD document

XSD: elemente

```
<xs:element name="student" type="studentType"/>
```

```
<xs:element name="firmenzusatz" type="firmenzusatzType" minOccurs="0"/>
```

```
<xs:element name="ShoppingCartItem" type="xs:string" maxOccurs="10"/>
```

→ Default minOccurs = 1 → Feld ist Pflicht

XSD:

```
<xs:element name="exam" type="xs:string"/>
```

XML:

```
<exam>Anwendungssysteme</exam>
```

Datentypen:

- xs: string
- xs: integer
- xs: Boolean
- xs: float
- xs: positiveInteger

Beispiel

Wir wollen, dass Feld nur bestimmte Werte annehmen kann

→ simpleType mit Restriction

```
<xs:simpleType name="meinTypName">
```

```
  <xs:restriction base="xs:string">
```

```
    <xs:facet1 value="..." />
```

```
    <xs:facet2 value="..." />
```

```
  </xs:restriction>
```

```
</xs:simpleType>
```



Enumeration: nur bestimmte Werte erlaubt

minLength / maxLength: Mindest-/Maximallänge

minInclusive / maxInclusive: Zahlenbereiche

length: Länge des strings

Pattern: Regulärer Ausdruck

-> BSP: `<xs:pattern value="[0-9]{5}" />` → Postleitzahl (5 Ziffern)

-> alle beliebigen Zahlen: `[0-9]`

-> alle beliebigen Zahlen, aber mindestens 3: `[0-9]{3,}`

-> alle beliebigen Zahlen, aber mindestens 2 und maximal 5: `[0-9]{2,5}`

-> alle beliebigen Zahlen, aber mit 0 anfangend: `0[0-9]`

-> alle beliebigen Zahlen, aber maximal 4: `[0-9]{0,4}`

Beispiele zu simple Type

Beispiel 1)

```
<xs:simpleType name="termType">  
  <xs:restriction base="xs:string">  
    <xs:enumeration value="summer term"/>  
    <xs:enumeration value="winter term"/>  
  </xs:restriction>  
</xs:simpleType>
```



Nur "summer term" oder "winter term" sind gültige Inhalte.

Beispiel 2)

```
<xs:restriction base="xs:integer">  
  <xs:minInclusive value="0"/>  
  <xs:maxInclusive value="50"/>  
</xs:restriction>
```



Nur ganze Zahlen zwischen 0 und 50 sind erlaubt.

complexType

- Sobald Element Kindelemente/ Attribute haben soll

```
<xs:complexType name="meinTypName">
```

```
<xs:sequence>
```

```
<xs:element name="kind1" type="irgendeinTyp"/>
```

```
<xs:element name="kind2" type="nochEinTyp"/>
```

```
</xs:sequence>
```

```
<xs:attribute name="meinAttribut" type="xs:string"/>
```

```
</xs:complexType>
```

Schlüsselwort

Bedeutung

```
<xs:sequence>
```

Alle Kinder müssen **in genau dieser Reihenfolge** vorkommen

```
<xs:choice>
```

Nur eines der definierten Kinder darf vorkommen

```
<xs:all>
```

Alle Kinder können vorkommen, aber **in beliebiger Reihenfolge**

Beispiel

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="student" type="studentType"/>

  <xs:complexType name="studentType">
    <xs:sequence>
      <xs:element name="firstName" type="xs:string"/>
      <xs:element name="lastName" type="xs:string"/>
      <xs:element name="age" type="xs:integer"/>
      <xs:element name="email" type="xs:string" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

XML:

```
<student>
  <firstName>Anna</firstName>
  <lastName>Müller</lastName>
  <age>22</age>
</student>
```

Zusatz – Attribute

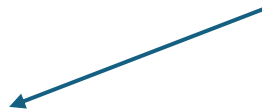
Für **Attribute** gibt es das Attribut **use**:

- required – muss vorhanden sein
- optional – darf fehlen (Standard)
- forbidden – darf nicht vorhanden sein

```
<xs:attribute name="ID" type="xs:positiveInteger" use="required"/>
```

XML:

Das ist ein Attribut



```
<student id="42">  
</student>
```

Ist immer root element

Eingeschränkte einfache Typen (kein Kindelement)

XSD document

```
<?xml ...>
<xs:schema ...>
  <xs:simpleType name="termType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="summer term"/>
      <xs:enumeration value="winter term"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="examType">
    <xs:sequence>
      <xs:element name="lecture" type="xs:string" maxOccurs="1"/>
      <xs:element name="term" type="termType" maxOccurs="1"/>
    </xs:sequence>
    <xs:attribute name="room" type="xs:string"/>
  </xs:complexType>
  <xs:complexType name="examListType">
    <xs:sequence>
      <xs:element name="exam" type="examType" maxOccurs="Unbounded"/>
    </xs:sequence>
  </xs:complexType>
  <xs:element name="exams" type="examListType"/>
</xs:schema>
```

xs:sequence / xs:choice / xs:all

```

<?xml ...>
<xs:schema ...>
  <xs:simpleType name="termType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="summer term"/>
      <xs:enumeration value="winter term"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="examType">
    <xs:sequence>
      <xs:element name="lecture" type="xs:string" maxOccurs="1"/>
      <xs:element name="term" type="termType" maxOccurs="1"/>
    </xs:sequence>
    <xs:attribute name="room" type="xs:string"/>
  </xs:complexType>
  <xs:complexType name="examListType">
    <xs:sequence>
      <xs:element name="exam" type="examType" maxOccurs="Unbounded"/>
    </xs:sequence>
  </xs:complexType>
  <xs:element name="exams" type="examListType"/>
</xs:schema>

```

XSL

```

<exams>
  <exam room="A101">
    <lecture>Anwendungssysteme</lecture>
    <term>summer term</term>
  </exam>
  <exam room="B202">
    <lecture>Datenbanken</lecture>
    <term>winter term</term>
  </exam>
  <exam>
    <lecture>Verteilte Systeme</lecture>
    <term>summer term</term>
  </exam>
</exams>

```

Aufgabe 2: XML Schema

Ordnen Sie die folgenden Fragmente aus XML-Schema- und XML-Dokumenten einander zu.
Hinweis: Zum Teil gibt es mehrere Möglichkeiten oder auch keine richtige Lösung.

```
<xs:element name="dataentry" type="data"/>
<xs:simpleType name="data" type="xs:string"/>
```

A, C, D

```
<xs:element name="dataentry" type="data"/>
<xs:simpleType name="data">
  <xs:restriction base="xs:string">
    <xs:maxLength value="6"/>
  </xs:restriction>
</xs:simpleType>
```

C, D

```
<xs:element name="dataentry" type="data"/>
<xs:simpleType name="data">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Monday"/>
    <xs:enumeration value="Tuesday"/>
    <xs:enumeration value="Wednesday"/>
    <xs:enumeration value="Thursday"/>
    <xs:enumeration value="Friday"/>
    <xs:enumeration value="Weekend"/>
  </xs:restriction>
</xs:simpleType>
```

D

A)
<dataentry>some value</dataentry>

C)
<dataentry>value</dataentry>

B)
<data>some value</data>

D)
<dataentry>Monday</dataentry>

```
<xs:element name="dataentry" type="data"/>
<xs:complexType name="data">
  <xs:choice>
    <xs:element name="value" type="xs:string" maxOccurs="1"/>
    <xs:element name="value2" type="xs:integer" maxOccurs="1"/>
  </xs:choice>
</xs:complexType>
```

H,J

```
<xs:element name="dataentry" type="data"/>
<xs:complexType name="data">
  <xs:sequence>
    <xs:element name="value" type="xs:string"
      minOccurs="0" maxOccurs="5"/>
    <xs:element name="value2" type="xs:integer"
      minOccurs="0" maxOccurs="5"/>
  </xs:sequence>
</xs:complexType>
```

F,H,I,J

F)
<dataentry>
 <value>some value</value>
 <value2>12345</value2>
</dataentry>

I)
<dataentry>
 <value2>123</value2>
 <value2>456</value2>
 <value2>789</value2>
 <value2>0</value2>
</dataentry>

G)
<data>
 <value>some value</value>
</data>

H)
<dataentry>
 <value2>12345</value2>
</dataentry>

J)
<dataentry>
 <value>Friday</value>
</dataentry>

Aufgabe 7 – XML-Schema

Erstellen Sie ein XML-Schema, welches das Adressfeld einer Firma mit Standort Deutschland beschreibt. Das XML-Schema soll die folgenden Informationen beschreiben. Vervollständigen Sie außerdem das untenstehende Beispiel, verwenden Sie dabei auch alle optionalen Elemente.

Element	Pflicht	Beschreibung
Firmenname	x	Text mit beliebiger Länge
Firmenzusatz		erlaubte Werte „GmbH“ und „AG“
Straße	x	Zeichenkette; außerdem Leerzeichen, Punkte und Zahlen erlaubt
Hausnummer	x	natürliche Zahl
Postleitzahl	x	besteht aus genau fünf Ziffern
Stadt	x	Zeichenkette aus Buchstaben
Telefonnummer		besteht aus den Feldern Vorwahl (fängt immer mit 0 an, zwischen 3 und 5 Ziffern lang) und Nummer (fängt nicht mit 0 an, mindestens 4 Ziffern)

Dieses Schema soll zum Beispiel so verwendet werden.

```
<?xml version="1.0" encoding="UTF-8"?>
<adresse>
  <firmenname>Apfel Computer</firmenname>
  <firmenzusatz>AG</firmenzusatz>
  <straße>Unter-den-Fichten</straße>
  ...
</adresse>
```

```
<xs:simpleType name="meinTypName">
  <xs:restriction base="xs:string">
    <xs:facet1 value="..."/>
    <xs:facet2 value="..."/>
  </xs:restriction>
</xs:simpleType>
```

Enumeration: nur bestimmte Werte erlaubt
minLength / maxLength: Mindest-/Maximallänge
minInclusive / maxInclusive: Zahlenbereiche
length: Länge des strings
Pattern: Regulärer Ausdruck

- > BSP: `<xs:pattern value="[0-9]{5}" />` → Postleitzahl (5 Ziffern)
- > alle beliebigen Zahlen: `[0-9]`
- > alle beliebigen Zahlen, aber mindestens 3: `[0-9]{3,}`
- > alle beliebigen Zahlen, aber mindestens 2 und maximal 5: `[0-9]{2,5}`
- > alle beliebigen Zahlen, aber mit 0 anfangend: `0[0-9]`
- > alle beliebigen Zahlen, aber maximal 4: `[0-9]{0,4}`

```
<xs:complexType name="meinTypName">
  <xs:sequence>
    <xs:element name="kind1" type="irgendeinTyp"/>
    <xs:element name="kind2" type="nochEinTyp"/>
  </xs:sequence>
  <xs:attribute name="meinAttribut" type="xs:string"/>
</xs:complexType>
```

Schlüsselwort	Bedeutung
<xs:sequence>	Alle Kinder müssen in genau dieser Reihenfolge vorkommen
<xs:choice>	Nur eines der definierten Kinder darf vorkommen
<xs:all>	Alle Kinder können vorkommen, aber in beliebiger Reihenfolge

Protobuf

- Binäres Serialisierungsformat
- -> Schneller als JSON und XML, aber nicht Menschenlesbar
- Ablauf:
 1. .proto Datei schreiben (vergleichbar mit XSD)
 2. Protobuf-Compiler in IntelliJ generiert daraus Java-Klassen
 3. benutzen diese generierten Klassen zum Schreiben und Lesen von Daten

```
message Person {  
    optional string name = 1;  
    optional int32 id = 2;  
    optional string email = 3;  
}
```


Listen und objekte

- Wir haben eine message für jede Datenstruktur die wir haben wollen
- Für Listen von Objekten benutzt man ein **Wrapper-Message**:

```
java_package = "de.mcc.mcc";  
option java_outer_classname = "SchoolProto";
```

```
message Student { //generierte Java Klasse heißt Student  
    optional int32 id = 1;  
    optional string first_name = 2;  
    optional string last_name = 3;  
    optional string email = 4;  
}
```

```
message SchoolClass {  
    optional string name = 1;  
    repeated Student students = 2;  
}
```

Tag-Nummer -> sorgt für
eindeutigkeit -> keine
Nummern Doppelt

Jedes Feld hat drei Teile:

Modifier:

- optional (darf fehlen)
- repeated (Liste, 0 oder mehr Einträge)

Typ: string, int32, bool, float, ... oder ein anderer
message-Typ

Protobuf

```
package org.example;  
option java_outer_classname = "SchoolProto";
```

```
message Student { //generierte Java Klasse heißt Student  
    optional int32 id = 1;  
    optional string first_name = 2;  
    optional string last_name = 3;  
    optional string email = 4;  
}
```



```
Student s = Student.newBuilder()  
    .setId(1)  
    .setFirstName("Anna")  
    .setLastName("Müller")  
    .setEmail("anna@example.com")  
    .build();
```

```
message SchoolClass {  
    optional string name = 1;  
    repeated Student students = 2;  
}
```



```
SchoolClass sc = SchoolClass.newBuilder()  
    .setName("Informatik")  
    .addStudents(s) //fügen Student einzeln ein  
    .build();
```

Aufgabe 9 – Protobuf

Benutzen Sie die IntelliJ Vorlage und erstellen Sie im Ordner `src/main/proto` eine Datei `users.proto`, mit der sie die Daten aus Aufgabe 11 speichern können. Schreiben Sie dann eine `main`-Methode, die die User in eine Datei schreibt (z. B. `user.bin`) und eine andere `main`-Methode, die diese Datei einliest und die User auf der Konsole ausgibt.

id	name	address
1	Müller	Englerstraße 11
2	Meyer	Daimlerweg 50
3	Kunz	Fährweg 12
4	Schmidt	Ettlinger Straße 90

```
package org.example;  
option java_outer_classname = "SchoolProto";
```

```
message Student { //generierte Java Klasse heißt Student  
    optional int32 id = 1;  
    optional string first_name = 2;  
    optional string last_name = 3;  
    optional string email = 4;  
}
```



```
Student s = Student.newBuilder()  
    .setId(1)  
    .setFirstName("Anna")  
    .setLastName("Müller")  
    .setEmail("anna@example.com")  
    .build();
```

```
message SchoolClass {  
    optional string name = 1;  
    repeated Student students = 2;  
}
```



```
SchoolClass sc = SchoolClass.newBuilder()  
    .setName("Informatik 1")  
    .addStudents(s) //fügen Student einzeln ein  
    .build();
```

GitHUb –
Code kommt
später

